Mesh partitioning and node ordering for FESOM2/Kokkos: measurements on CPU and GPU, and a procedure for adopting a new partition

Campaign report

12 August 2026 — Levante (128-core CPU nodes, 4×A100 GPU nodes; GH200 testbed)

TL;DR

- Repartitioning makes the GPU model step **8–14 % faster** (DARS on 64 GPUs: −14.3 %; a −19.7 % arm exists there but failed the accuracy test). The gain comes from one METIS option, `MINCONN`, which FESOM's `PartGraphRecursive` call silently ignores; switching to `PartGraphKway` is a few lines.
- At three of the four GPU points the winning partition is out of the accuracy class in exactly one statistic, the SSH rms (Section 6); adopting those gains is an explicit decision, not a test outcome.
- CPU runs gain **4–8 %**, from external engines (KaMinPar, Mt-KaHyPar) and load-balance slack instead; the two backends need different partitions of the same mesh.
- **Some new partitions crash the model** — eight caught, one produced by the unmodified stock recipe. Nothing computable from the partition predicts which. Every new partition must complete 3,000 steps at its target rank count before use; on failure, regenerate with a new random seed.
- Hilbert renumbering pays only on CORE2 (−5.8 % combined with repartitioning).

1. Summary

FESOM2 reads its domain decomposition from files, so a better partition can be adopted without touching model code or affecting any other configuration. We generated several hundred candidate partitions of four meshes with METIS under settings FESOM does not currently use, with three external partitioning engines, and with two node renumberings, and timed them against the shipped partitions on both backends of the C++/Kokkos port.

Three results carry the report. First, the shipped partitions cost the GPU backend up to 20 % of the model step, and most of that is recovered by a single METIS option, `MINCONN`, which no FESOM partition has ever had active: the partitioner calls `METIS_PartGraphRecursive`, which accepts and ignores it. Of the measured speed, 8–14 % stands after the accuracy tests, with the two largest gains (DARS −14.3 %, NG5 −9.8 %) out of class in the SSH rms alone and therefore left as explicit adoption decisions. Second, the CPU backend gains less (4–8 %) and gains it from different settings, namely external engines and load-balance slack; the two backends are best served by different partitions of the same mesh. Third, and unexpectedly, a small fraction of newly generated partitions cause the model to diverge, regardless of which settings produced them. Eight such partitions were caught during the campaign, one of them generated by the unmodified shipped recipe. Because no property computable from the partition itself identifies the dangerous ones, any partition intended for production must first complete a stability run at its target rank count; regenerating a failed partition with a different random seed has so far always produced a sound replacement.

Meshes: CORE2 (127k surface nodes), fArc (638k), DARS (3.16M), NG5 (7.40M). All experiments are cold starts under JRA55 (1958) forcing from PHC climatology, at the largest time step at which a cold start of the given mesh is known to be stable: 1800 s on CORE2, 900 s on fArc, 120 s on DARS, 180 s on NG5.

2. Candidates

METIS settings. Switching the METIS call from `PartGraphRecursive` to `PartGraphKway` makes three previously ignored options effective: the communication-volume objective, contiguity of the parts (`CONTIG`), and `MINCONN`, which minimises the largest number of neighbouring subdomains any part has. We swept these together with load-balance slack (`UFACTOR` of 10, 30 and 100), a family of vertical weights $w = a + \mathrm{nlev}$ with a between 0 and 100, two-level hierarchical partitions, and the METIS random seed. The Recursive call is not an oversight in the original code: `fort_part.c` documents that it produced the better partition on a test mesh, and notes which options it ignores. The comparison, however, predates the GPU backend.

External engines. KaHIP, KaMinPar and Mt-KaHyPar, each given the same node-weighted dual graph and the same weight sweep.

Node renumbering. Reordering the mesh files themselves along a Hilbert space-filling curve, and alternatively by reverse Cuthill–McKee, to improve memory locality. Unlike a partition, a renumbering changes the mesh files, so every reference solution pinned to the old numbering must be re-established.

A second architecture. The best GPU settings were re-timed on DKRZ's GH200 nodes (dolpung), whose interconnect cannot register GPU memory; halos there are staged through pinned host buffers, which prices communication differently than the A100 fabric does.

3. Measurement procedure

Timing runs place all candidates in a single job allocation and interleave repetitions, so that node-to-node and time-of-day variation affects every candidate alike. Each run integrates 300 steps; we quote the best of N repetitions, as a percentage of the shipped partition's time per step. The mesh-definition files are verified byte-identical across candidates before the job starts, so the candidates differ in the decomposition only.

A timing result is accepted only after two further tests.

Stability. The candidate must complete 3,000 steps (roughly two model months) at the rank count at which it will be used. This test exists because it fails in practice: a DARS partition that had won its timing comparison by 4.5 % diverged between step 150 and step 3,000 while the shipped partition ran cleanly.

Accuracy. Any change of decomposition reorders floating-point reductions, and the resulting round-off differences grow chaotically at fronts, so a new partition cannot be required to reproduce the reference solution. It can be required to be indistinguishable from an ordinary repartitioning. We therefore regenerate the shipped recipe with three or more different random seeds (verifying that each is a genuinely different partition), run twenty steps with each, and demand that the candidate's temperature, salinity and sea-surface-height rms differences from the reference fall inside the range spanned by those control partitions. The rms envelope is the criterion throughout; quantiles of the difference fields are recorded as context but do not decide.

Certified partitions are packaged by a tool that refuses any decomposition lacking a recorded clean 3,000-step run at its exact rank count.

4. Speed

Figure 1 shows the best candidate at each mesh and rank count; Table 1 records the settings and the state of the evidence behind each bar. Every gain that was re-measured over 3,000 steps came back equal to or larger than its 300-step value (Fig. 2b; for example −18.6 to −19.7 % on DARS GPU), so the short timing runs are, if anything, conservative.

4.1. What the two backends pay for

Across the nine groups of candidates that were timed, the number of communication partners per rank is the only precomputable quantity that correlates positively with GPU step time. Every

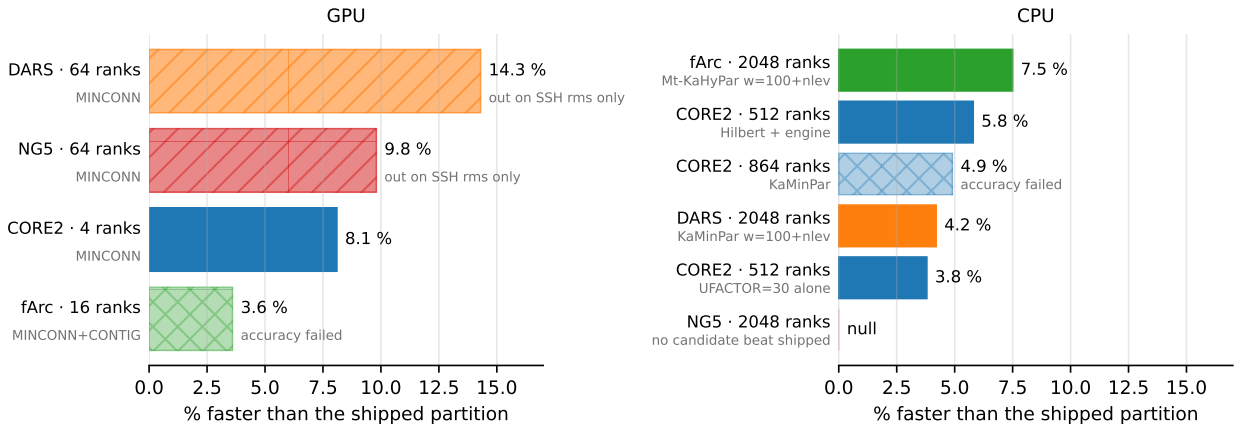

Figure 1: Best candidate partition per mesh and rank count, as a percentage faster than the shipped partition. Gray text under each label is the setting that produced the candidate. Solid bars are certified (stability and accuracy both passed); single-hatched bars are in class on everything except the SSH rms, with adoption an open decision; cross-hatched bars failed the accuracy test. Mesh colors follow the project's figure conventions and recur in Figs. 2 and 3.

mesh	backend, ranks	gain	setting	stability	accuracy
DARS	GPU, 64	−**14.3** %	`MINCONN`	passed	ssh rms +11 %, rest in class[a]
NG5	GPU, 64	−**9.8** %	`MINCONN`	passed	ssh rms +8 %, rest in class[a]
CORE2	GPU, 4	−**8.1** %	`MINCONN`	passed	passed
fArc	GPU, 16	−3.6 %	`MINCONN+CONTIG`	passed	**failed**; not recommended
fArc	CPU, 2048	−**7.5** %	Mt-KaHyPar w=100+nlev	passed	passed (4 controls)
CORE2	CPU, 512	−**5.8** %	Hilbert renumbering + engine	passed	passed
DARS	CPU, 2048	−**4.2** %	KaMinPar w=100+nlev	passed	passed
CORE2	CPU, 864	−4.9 %	KaMinPar	passed	**failed** (salinity); not recommended
CORE2	CPU, 512	−3.8 %	`UFACTOR=30` alone	passed	passed
NG5	CPU, 2048	null	—	—	—

Table 1: Best candidate per mesh and rank count, relative to the shipped partition, at the time step given in Section 1. [a]The DARS and NG5 GPU accuracy tests were still in the queue when this report was written; the first DARS pass fell just outside a three-control envelope and is being re-judged against five controls (Section 6).

volume-like quantity — edge cut, halo bytes, replicated elements — correlates negatively: on the GPU, candidates that ship more data are faster when they talk to fewer neighbours. `MINCONN` is the METIS objective that acts on partner count, which is why it wins on the GPU and why the win remained invisible while the Recursive call ignored the option.

The size of the GPU gain across meshes follows the fractional reduction in the *maximum* partner count (Fig. 2a); the mean partner count is reduced by a similar fraction at all three points and therefore explains none of the ordering. That the maximum is the relevant statistic is expected, since each step waits for the slowest rank. The relation holds within one machine; Section 4.3 shows that its coefficient does not transfer across interconnects.

On the CPU, no precomputed quantity orders the candidates: for load imbalance, edge cut and partner count alike, the rank correlation with step time straddles zero across the timed groups. CPU candidates therefore have to be timed. What does generalise is that an external engine won at every CPU point with 512 or more ranks, and that relative to `MINCONN` alone, adding `CONTIG` and slack gains five percentage points on DARS GPU and loses one to two on DARS CPU: the same options move the two backends in opposite directions.

4.2. Node renumbering pays only on CORE2

Hilbert renumbering is worth −1.2 to −2.4 % on CPU and −5.0 % on a single GPU node on CORE2, and combines almost additively with repartitioning (−5.8 % at 512 ranks). The other meshes gain

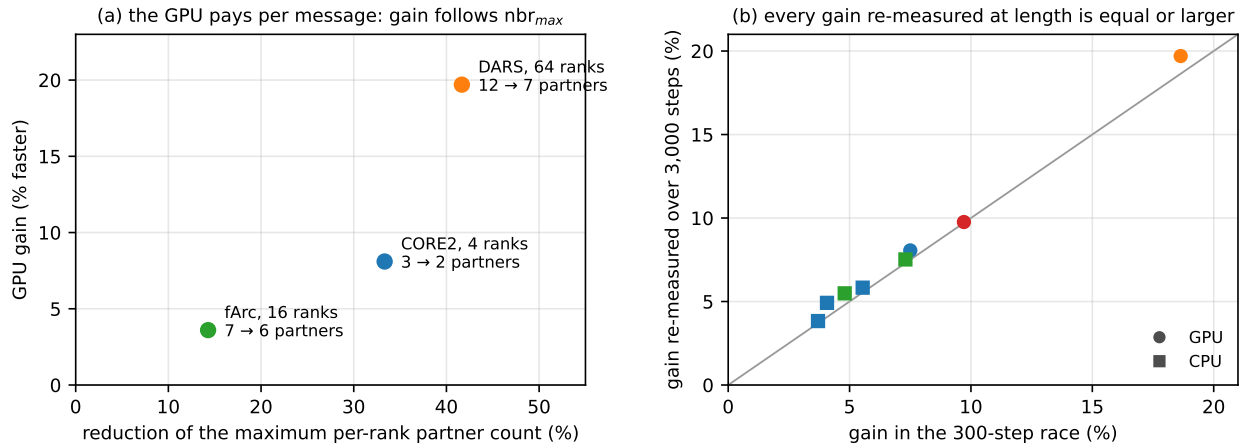

Figure 2: (a) GPU gain against the fractional reduction of the maximum per-rank communication-partner count, annotated with the counts themselves. (b) Gain measured in the 300-step timing run against the same candidate's gain re-measured over 3,000 steps, for the eight candidates measured both ways; the line marks equality, circles are GPU points, squares CPU, and colors identify the mesh as in Fig. 1.

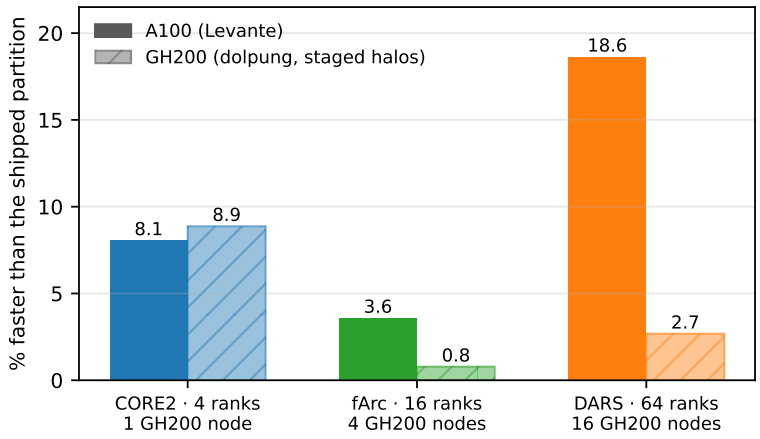

Figure 3: The three GPU settings re-timed on GH200 (hatched) against their A100 values (solid), with the GH200 node count under each pair. The GH200 values for fArc and DARS are medians, because the shipped partition was the noisiest configuration in those jobs and the best-of-N estimator is then biased; the fArc value is indistinguishable from zero at the measured noise level.

nothing, because their element-gather locality as shipped is already 88 %, against 27.6 % for CORE2. Since a renumbering forces all pinned references to be re-established (Section 2), we recommend it only where that cost has been accepted; the repartitioning gain requires no such decision.

4.3. The second architecture

On a single GH200 node the CORE2 result reproduces slightly above its A100 value (−8.9 against −8.1 % with `MINCONN`, two independent jobs agreeing within 0.6 percentage points), so the effect is not specific to A100 hardware. It did not, however, survive the move to several nodes (Fig. 3): a prediction, registered before the job ran, that DARS on 64 GH200 ranks would exceed its A100 gain of −18.6 % failed, with the measured effect between −1 and −3 %, and fArc on 4 nodes shows no effect at all at five repetitions. On this machine the per-message cost that `MINCONN` removes evidently resides in the staged intra-node path, not in the inter-node fabric. Multi-node gains should therefore be measured on each interconnect rather than transferred. The same experiments showed that quoting the best of N repetitions is biased whenever the reference partition is noisier than the candidates, as it consistently was there; medians are reported alongside in that situation.

5. Some repartitioned meshes crash the model

Eight of the partitions generated during this campaign cause the model to diverge: on NG5, `MINCONN`-family candidates at both 2,048 and 64 ranks and a slack-only candidate at 64; on DARS, `MINCONN`-family candidates at 2,048 ranks; and on DARS at 2,048 ranks one partition produced by the *unmodified shipped recipe* with a different random seed, which diverged within fourteen steps. The failure is always local: one region runs away while the global diagnostics remain normal for tens of steps.

Three properties of these failures matter for practice.

First, they carry no offline signature. In the NG5 group, the partitions that later diverged scored better than the survivors on every precomputed quantity we examined, including load imbalance, edge cut and the number of isolated nodes.

Second, short tests do not find them. A five-step run passes on a partition that diverges at step 71, and the 150-step timing runs passed on a partition that diverged before step 3,000. The 3,000-step requirement of Section 3 caught every failure listed above; nothing shorter would have.

Third, the failure belongs to the individual partition, not to the settings that produced it. On both meshes where a winning candidate diverged, regenerating it with the same settings and a different METIS seed gave a partition that passed — the full 3,000-step test on DARS, the 150-step timing run on NG5 — in one case with a worse edge cut than the partition that had failed. Conversely, the settings of the DARS GPU winner produce a sound partition at 64 ranks and a fatally unstable one at 2,048.

Since the shipped recipe itself produced one of the eight, the shipped partitions in production differ from the failures only in having been exercised. The practical conclusion is that the stability test is a property of adopting *any* new partition, not a surcharge on the new options.

6. Accuracy

The control-envelope test of Section 3 discriminated in both directions. At CORE2, 4 GPU ranks, the winning candidates lie below every control on all three fields; at DARS, 2,048 CPU ranks, KaMinPar lies below every control on temperature and salinity. Both are certified. Conversely, the fArc GPU candidates lie above a four-control temperature envelope, and at CORE2, 864 ranks, KaMinPar's salinity rms is 24 % above a five-control envelope; both points are speed improvements that we nevertheless do not recommend. The controls themselves span a factor 1.05 on DARS and 3.9 on CORE2 at 864 ranks, so no fixed rms threshold could replace the per-mesh envelope.

One systematic effect complicates the sea-surface-height comparison. The barotropic solver stops at a relative residual, so a partition on which it converges faster stops earlier and lands measurably further from the reference. Partitions that improve convergence — which the `MINCONN` family does — are thereby penalised in exactly one field. The effect is visible at CORE2 (the candidate with the fewest solver iterations has the largest SSH difference) and again on DARS at 64 ranks, where all three candidates converge faster than all three controls. We record it as context; the envelope criterion stands.

7. Recommendations

For FESOM2 upstream, in order of value:

1. Call `METIS_PartGraphKway` in `fort_part.c`, so that `MINCONN`, `CONTIG` and the volume objective reach METIS, and expose the partitioner options as run-time switches rather than compile-time constants.

2. Adopt the stability requirement together with the options: every newly generated partition, whatever produced it, runs 3,000 steps at its target rank count and the mesh's cold-start time step before production use, and a failure is answered by regenerating with a new seed.

3. Rebuild the isolated-node repair in `check_partitioning`, which currently considers a single neighbour as reassignment candidate, and update METIS to 5.2.1.

On the machines measured here, the settings to reach for are `MINCONN` for GPU runs, with `CONTIG` and slack decided per configuration by a timing run, and an external engine (KaMinPar or Mt-KaHyPar, $w = 100 + \text{nlev}$) or `UFACTOR=30` for CPU runs. Hilbert renumbering is worthwhile on CORE2-class meshes where re-establishing the references has been accepted. Offline partition quality metrics are useful for designing candidates and useless for accepting them; acceptance is the stability and accuracy tests.

8. Status

Every gate has run; no measurement is pending. Two adoption decisions remain open, both of the same shape: `MINCONN` at DARS 64 ($-14.3\,\%$) and at NG5 64 ($-9.8\,\%$) are outside the accuracy class in the SSH rms alone. Under the fixed criterion neither is certified; adopting either is a documented decision rather than a test outcome. The certified partitions are packaged: four evidence-checked mesh directories under `mesh_m11_certified/` carry the CORE2, fArc and DARS winners with their run records. The upstream pull request is the remaining engineering step. Whether an interconnect with working GPU–direct communication prices messages like the A100 fabric or like the GH200 testbed is open, and JUPITER should answer it by measurement.

Evidence trail: `docs/PARTITIONING_M11.md` *(Findings 1–45, with job ids for every number quoted here); operational summary:* `docs/M11_RECOMMENDATION.md`*; full table of timing runs:* `scripts/m11_harvest_races.py -best`*.*